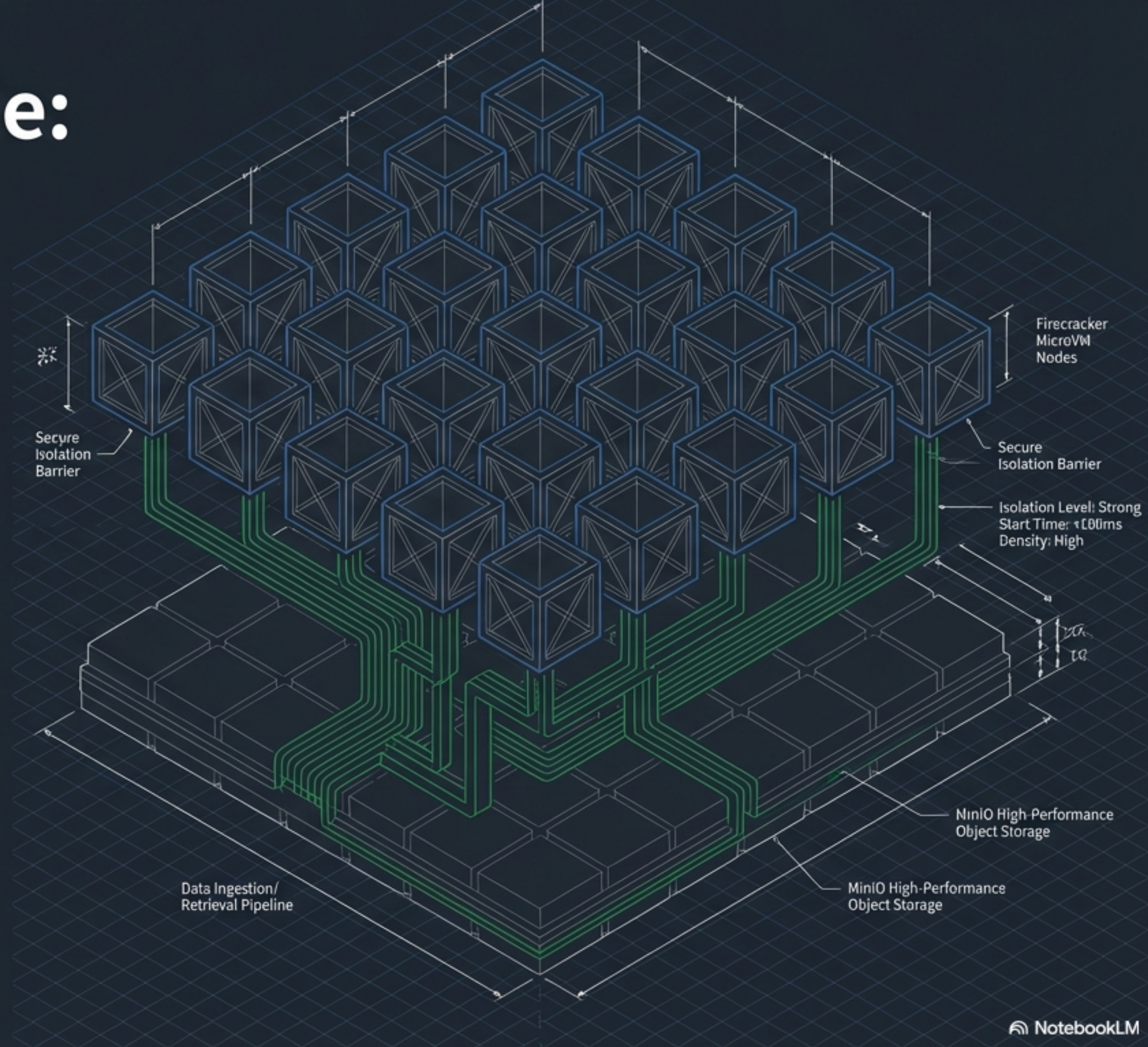


Project OpenCode: 企業級 AI Agent 基礎設施架構與 驗證報告

基於 Firecracker 與 MinIO
的高密度、強隔離、毫秒
級啟動架構決策評估



[CONFIDENTIAL / EXECUTIVE BRIEFING]

執行摘要：架構已驗證，建議持續推進 Firecracker 路線

針對「成千上萬個動態 AI Agent 並發執行」的需求，POC 數據證實 Firecracker 結合 MinIO S3 是唯一能同時滿足極致隔離與規模化的方案。強烈建議不應轉向傳統 K8s。

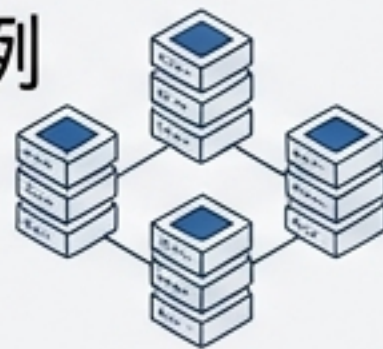
Risk (風險)

硬體級隔離 (Hardware-Level KVM)。徹底根絕 **CVE-2025-53773** 等 **Prompt Injection** 導致的 **內核逃逸風險**。



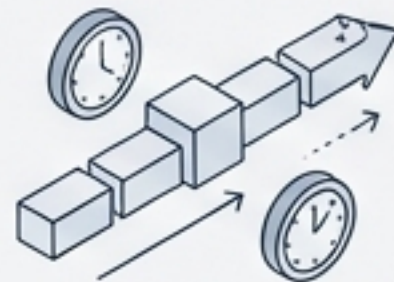
Cost & Scale (成本與規模)

極低開銷。POC 實測單一 VM 記憶體開銷僅 **~19.52 MB**，單節點安全乘載 **1000+** 個併實例在數學與實務上均完全可行。

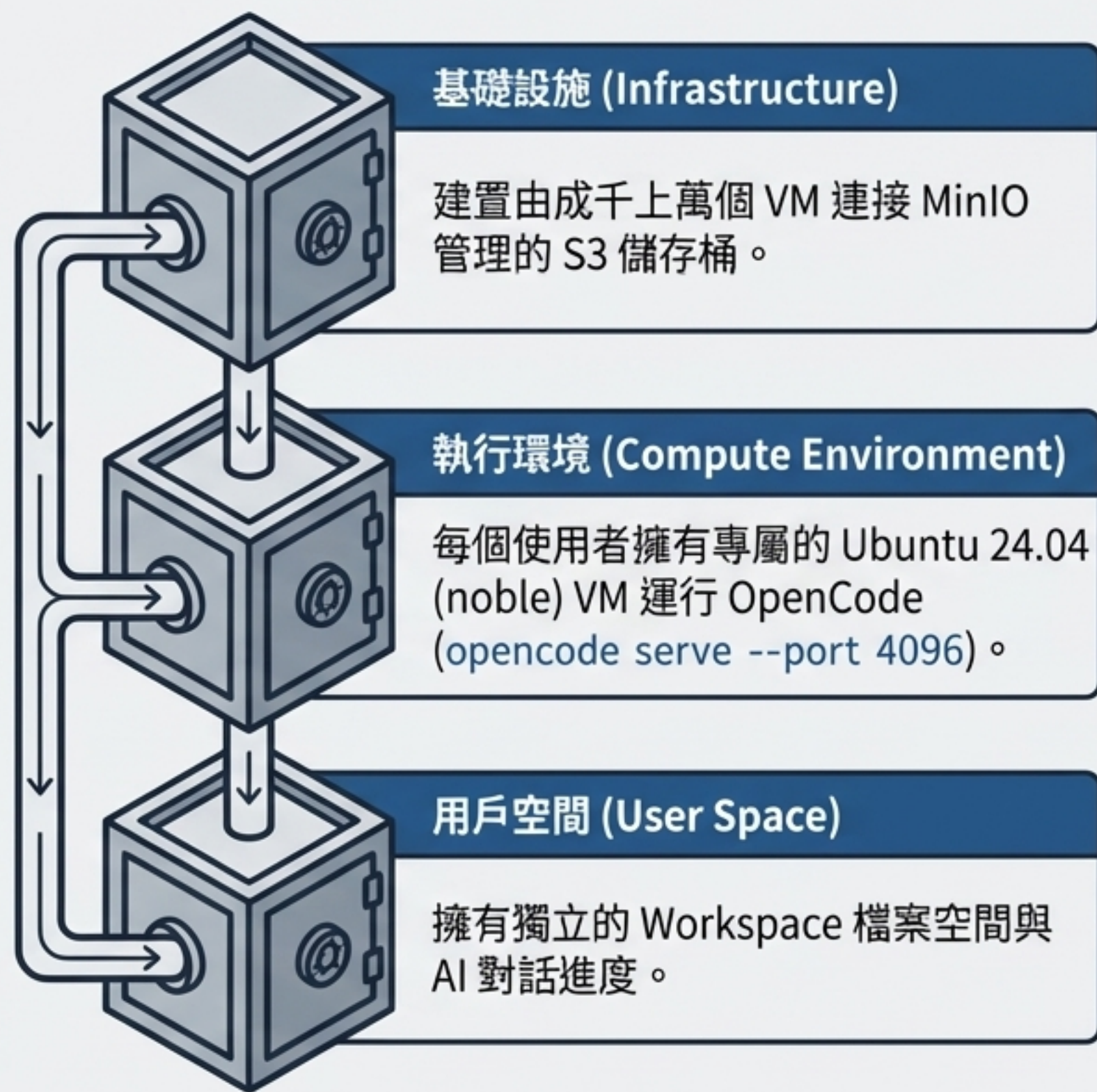


Performance (效能)

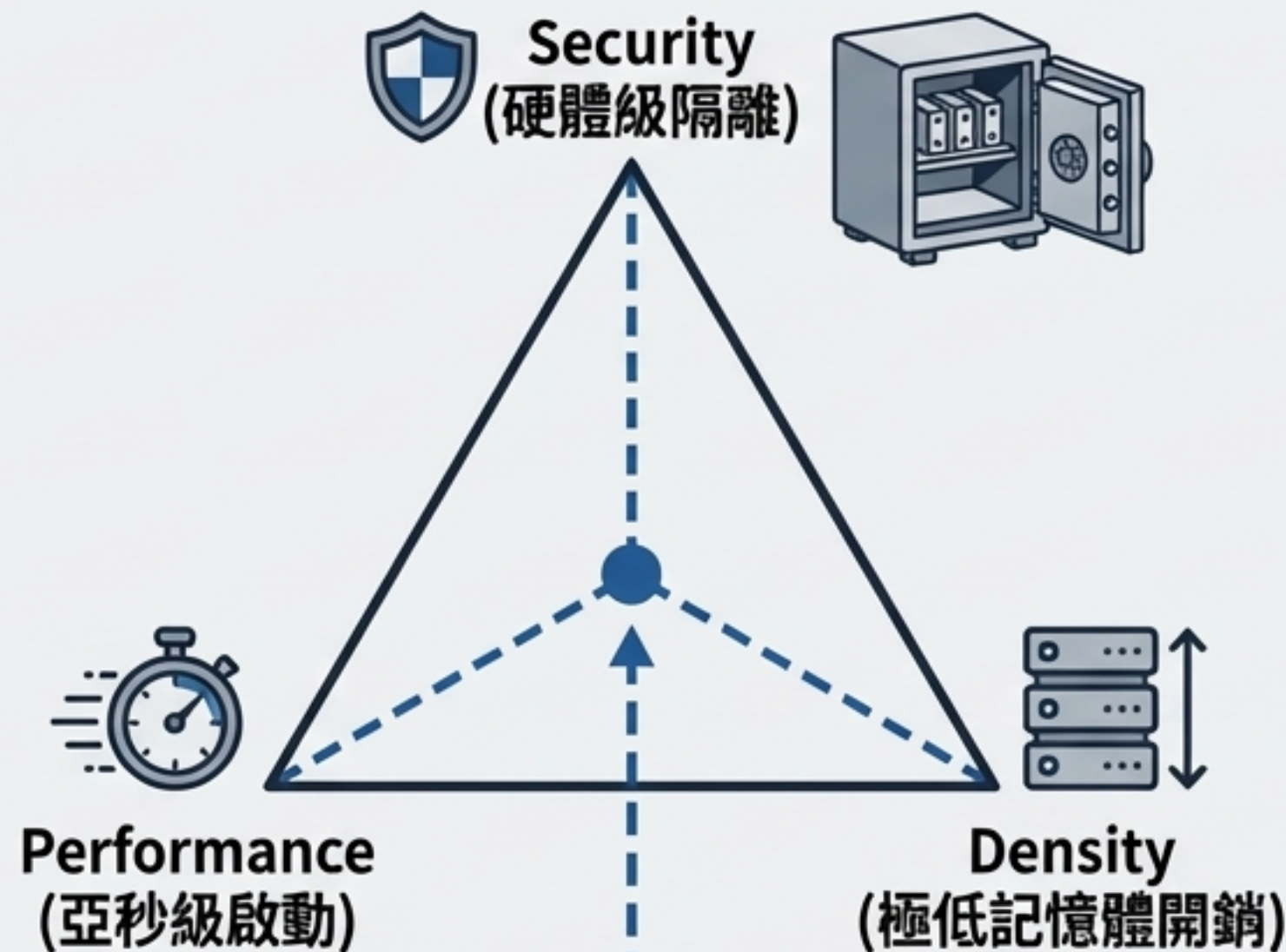
毫秒級冷啟動。API 指令延遲平均僅 **141.50ms**，P99 延遲穩定在 **215.90ms**，確保流暢的 AI 對話體驗。



目標願景與基礎設施的三難困境



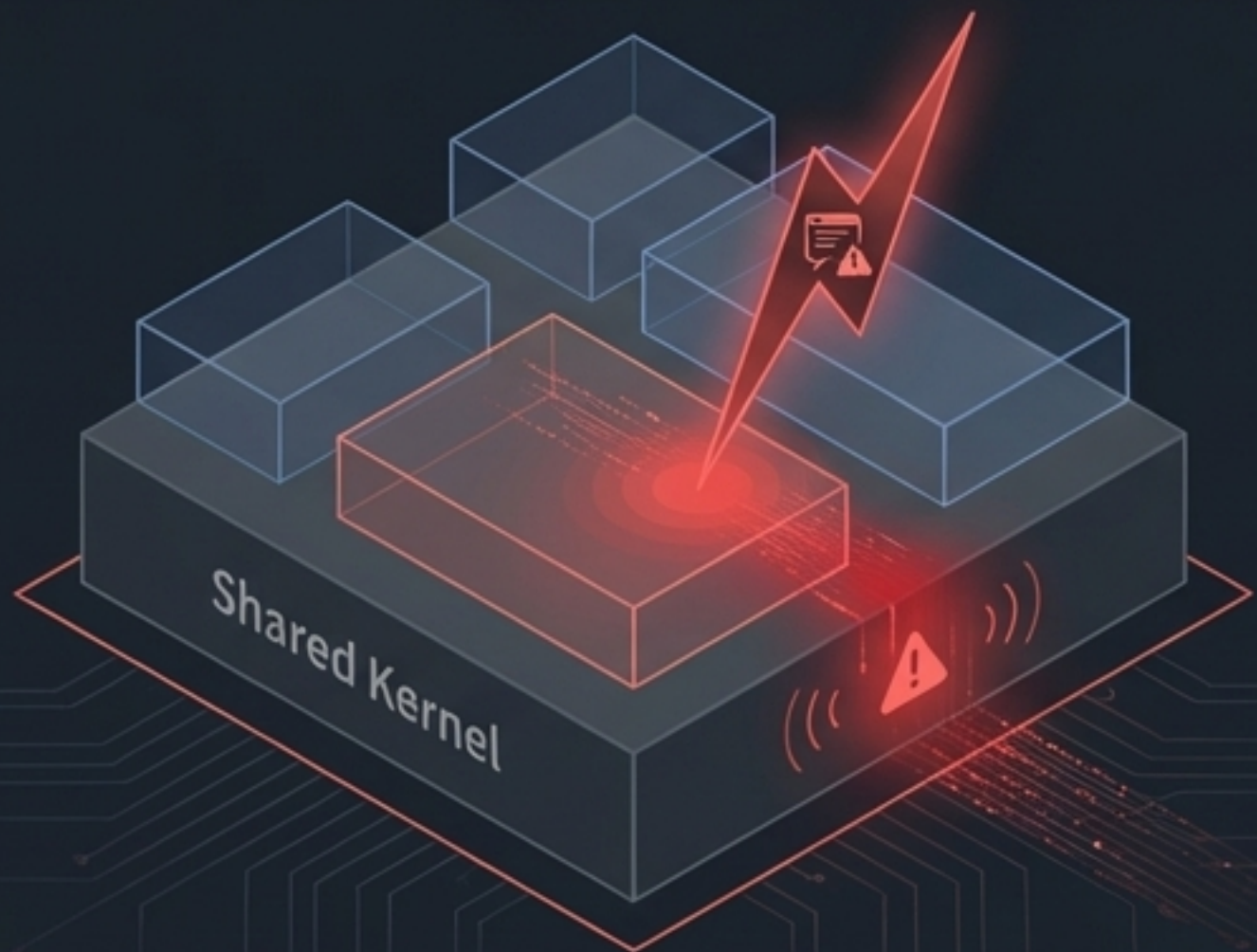
基礎設施三難困境 (Infrastructure Trilemma)



傳統機制（如標準容器）只能妥協其中一項。
我們的挑戰在於打破此困境，尋求位於完美幾何中心的解決方案。

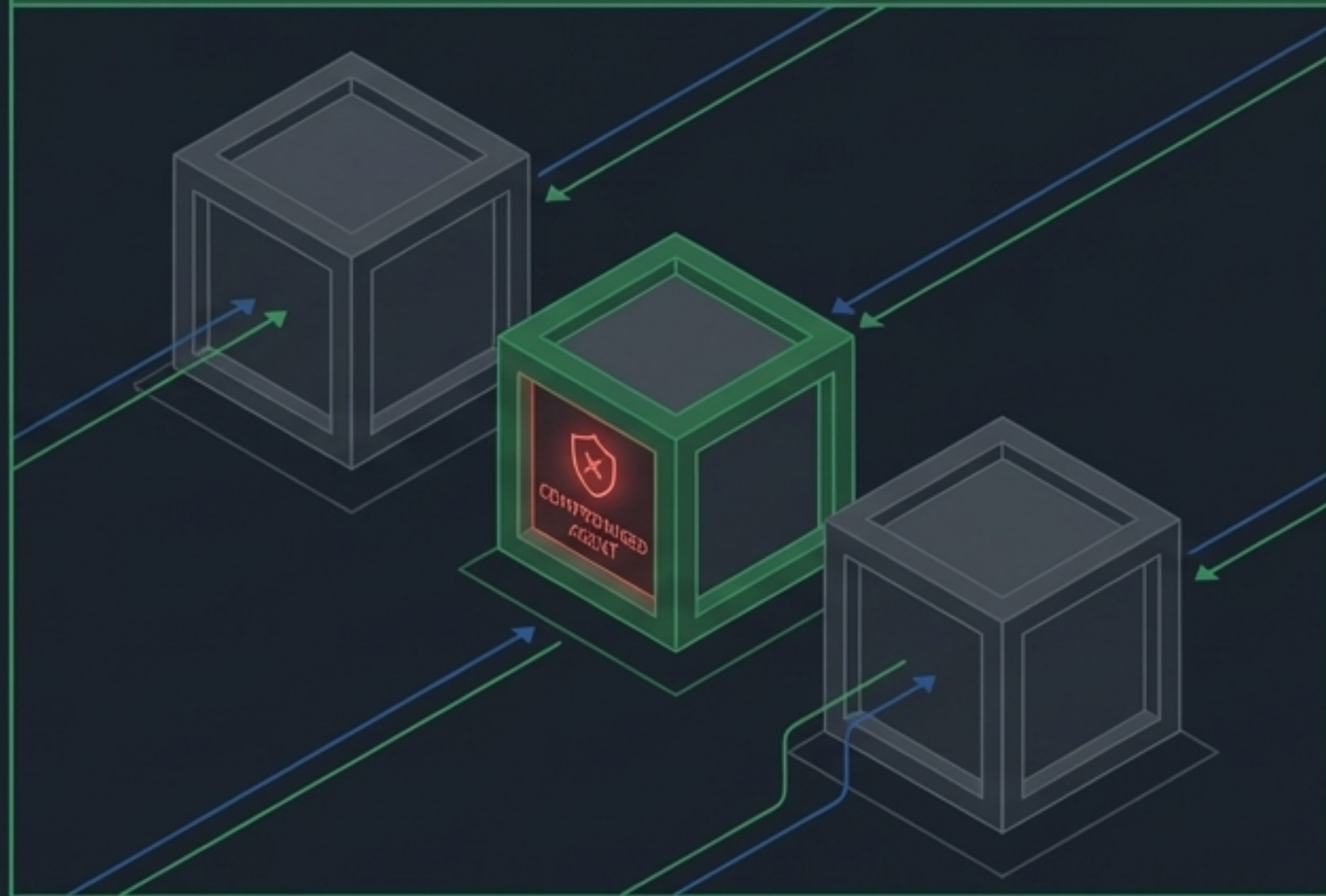
為什麼業界標準的 K8s 與 Docker 無法勝任 AI 沙盒？

The K8s & Docker Illusion



共享核心的致命傷。AI Agent 具有執行未知程式碼與決策的特性。一旦遭遇 Prompt Injection (如 GitHub Copilot 的 CVE-2025-53773 漏洞)，攻擊者可輕易進行提權 (Privilege Escalation) 與橫向移動。K8s/Docker 的命名空間與 cgroups 無法提供足夠的惡意程式碼防護。

The Hardware Vault



真正的硬體級隔離。我們需要的是為每一段 AI 程式碼分配獨立的 Guest Kernel。即使單一 Agent 被完全攻陷，爆炸半徑也嚴格侷限在該虛擬機內。

隔離技術評估矩陣：尋找 AI 沙盒的最佳引擎

技術	🕒 冷啟動	🛡️ 隔離強度	💻 記憶體開銷	AI 沙盒適用度
Docker / K8s	🕒 ~50ms	🛡️ 低 (共享內核)	極低	不合格 (高風險)
gVisor (Google)	🕒 ~100-150ms	中 (系統調用攔截)	中 (+30-50MB)	中等
Kata Containers	🕒 200-600ms	高 (輕量級 VM)	中	良好，但啟動偏慢
Firecracker (AWS)	🕒 ~125ms	最高 (KVM 硬體級)	極低 (<5MB Base)	最佳 (無伺服器首選)

Firecracker 剔除了傳統 VM 繁雜的虛擬硬體（僅保留 virtio-net, blk, vsock 等 5 個必要設備），在「強隔離」與「極速啟動」之間取得最完美的平衡。

技術象限：Firecracker 獨佔「高安全 x 高速度」魔術象限



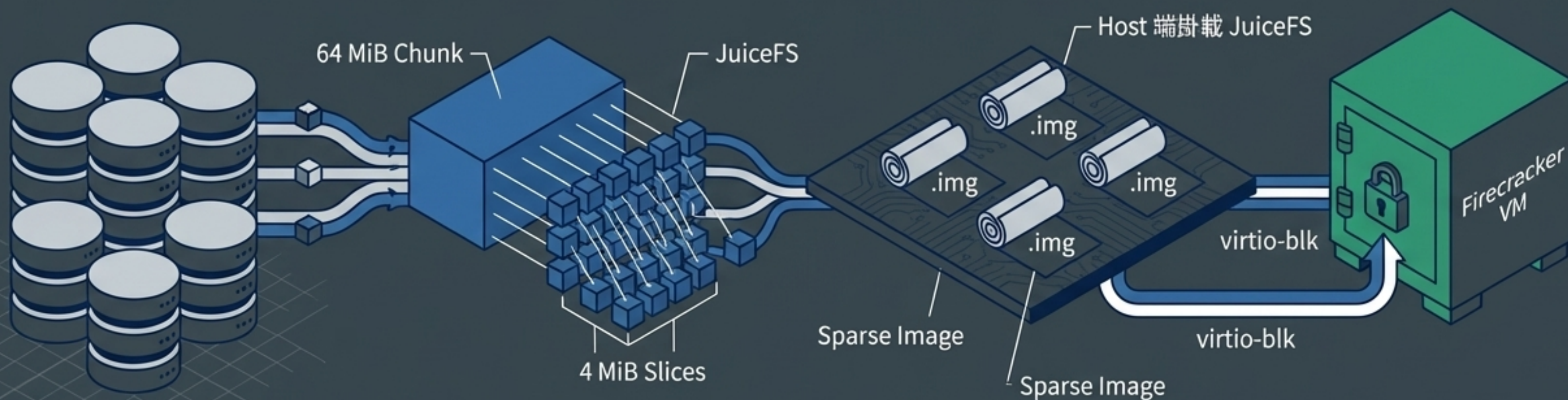
解耦架構藍圖：連結 S3 物件儲存與高速微虛擬機

Layer 1 (Storage Plane):
MinIO S3 集群

Layer 2 (The Bridge):
JuiceFS 分散式檔案系統
(64 MiB Chunks / 4 MiB Slices)

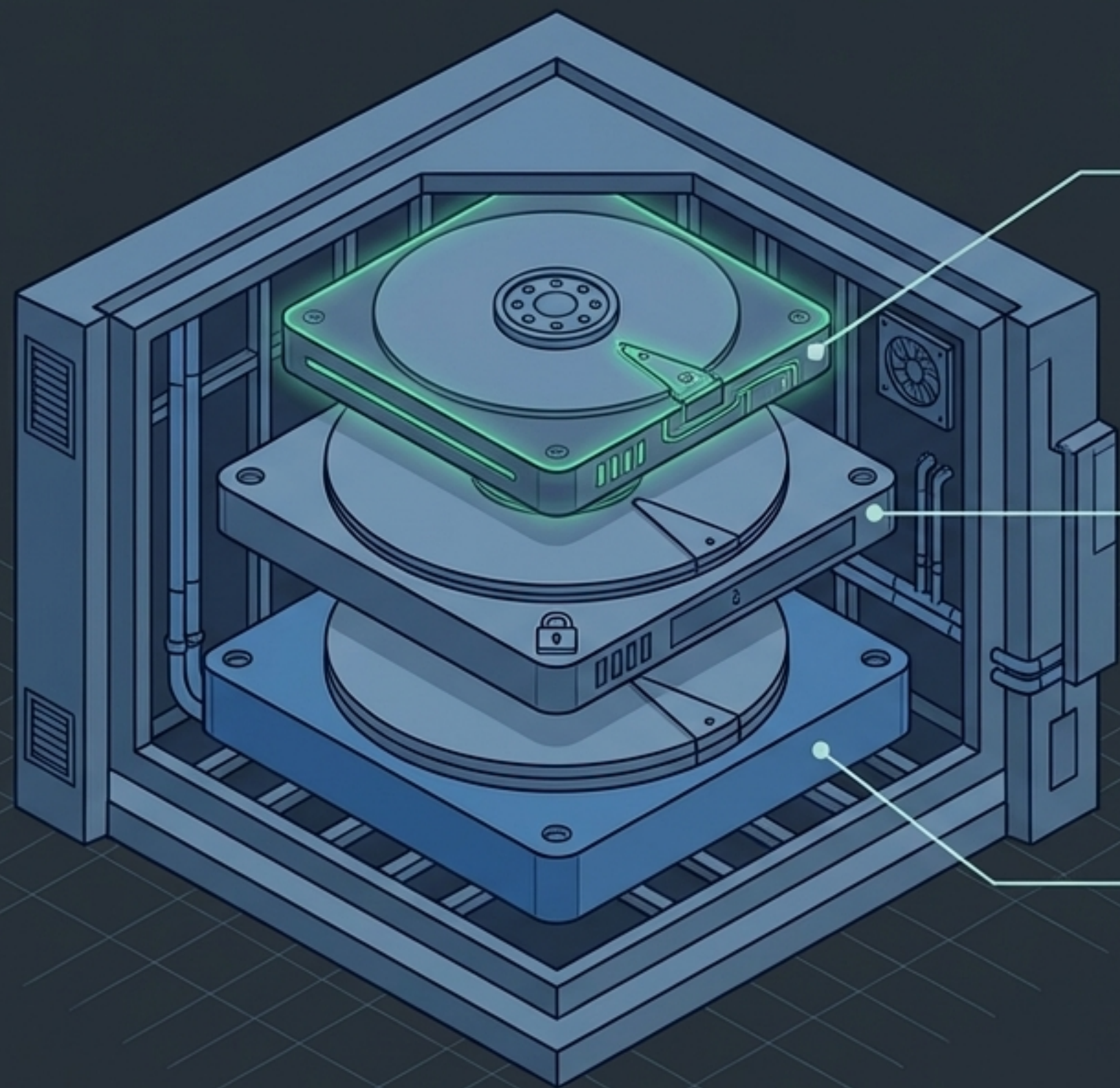
Layer 3 (Host Plane):
Host 端掛載 JuiceFS
(稀疏映像檔 Sparse Image)

Layer 4 (Compute Plane):
Firecracker VM
(virtio-blk 掛載)



這個解耦架構意味著資料雖然存在遠端 S3，但 VM 感受到的卻是本機端的高速區塊設備 (Block Device)，兼顧了擴展性與效能。

解剖 AI 工作空間：多層級的儲存映射策略



Layer 3: User Workspace (讀寫 / RW)

掛載點：/home/<user>/workspace 與 project/skills
內容：專屬使用者的動態檔案空間與 AI 對話進度。
機制：動態掛載難題。

Layer 2: Shared Skills (唯讀 / RO)

掛載點：~/.config/opencode/skills
內容：平台共用的基礎能力腳本。
機制：Block Mount (virtio-blk) 的獨立映像檔。

Layer 1: Base RootFS (唯讀 / RO)

掛載點：/
內容：Ubuntu 24.04 (noble) OS, OpenCode CLI, JS/Python 工具鏈。
機制：Block Mount (virtio-blk)。確保系統底層不被竄改，多個 VM 可安全共用。

動態檔案分享機制評估：為何選擇 SSHFS？

機制	效能/特性	即時雙向同步 (Live-Sync)	複雜度
Block Mount (virtio-blk)	最高、極穩定。但掛載後為獨佔或唯讀	否	低
9pfs (Plan 9)	支援檔案層級即時分享	是	中高 (受限於 virtio-9p 實作)
virtiofs	現代化高效共享系統	是	極高 (需 Host Daemon + Guest 支援)
SSHFS (採用方案)	透過 SSH 通訊協定掛載 Host 目錄	是	最低 (無須修改 Guest Kernel)

決策依據

靜態基礎層 (Static Base OS & Skills)

對於靜態、不頻繁變動的 Base OS 與 Shared Skills，採用 Block Mount (virtio-blk)。其高效與極穩定的特性確保了系統底層的完整性與多個 VM 間的安全共用，但限制為唯讀，適合基礎架構。

動態工作空間 (Dynamic User Workspace)

對於需要即時雙向同步的 User Workspace，在目前的基礎設施成熟度下，SSHFS 是最短路徑且最可落地的方案。無需修改 Guest Kernel 即可大幅降低維運複雜度，同時提供即時的檔案存取體驗。

架構大成：Stateless Compute + Stateful Storage

Stateless Compute

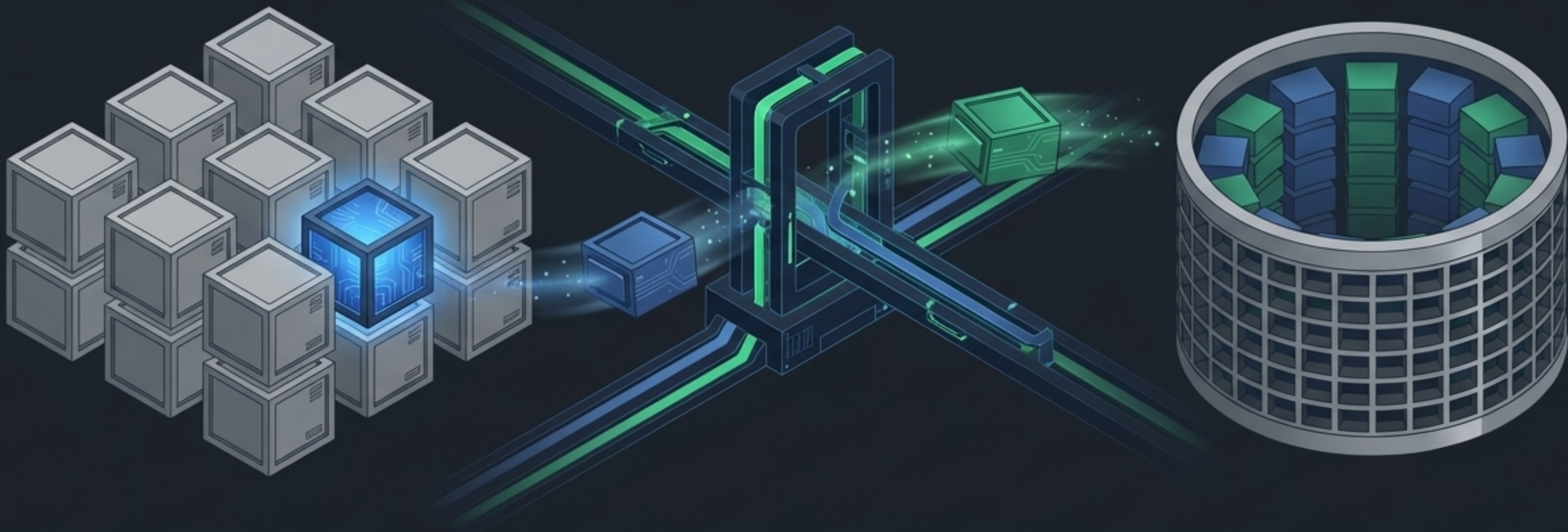
剛冷啟動完成的空白
Firecracker VM (無狀態)

The Injection API

毫秒間將專屬 Workspace
(SSHFS) 與 API Key 注入 VM

Stateful Storage

永久保存在 MinIO S3 中的
使用者動態 Workspace



成功的關鍵不在於 Firecracker 本身，而在於我們**成功解耦了運算與儲存**。VM 隨用隨丟（完全無狀態），保證系統絕對乾淨與安全；而使用者的資產與 AI 記憶則永久保存在 S3 中，實現**無限的橫向擴展** (Infinite Scale)。

POC 驗證與測試環境概覽

✓ **Host OS** : WSL2 / Linux 環境



✓ **Guest OS** : 完整建置 Ubuntu 24.04 (noble) base rootfs，內建 OpenCode API Server



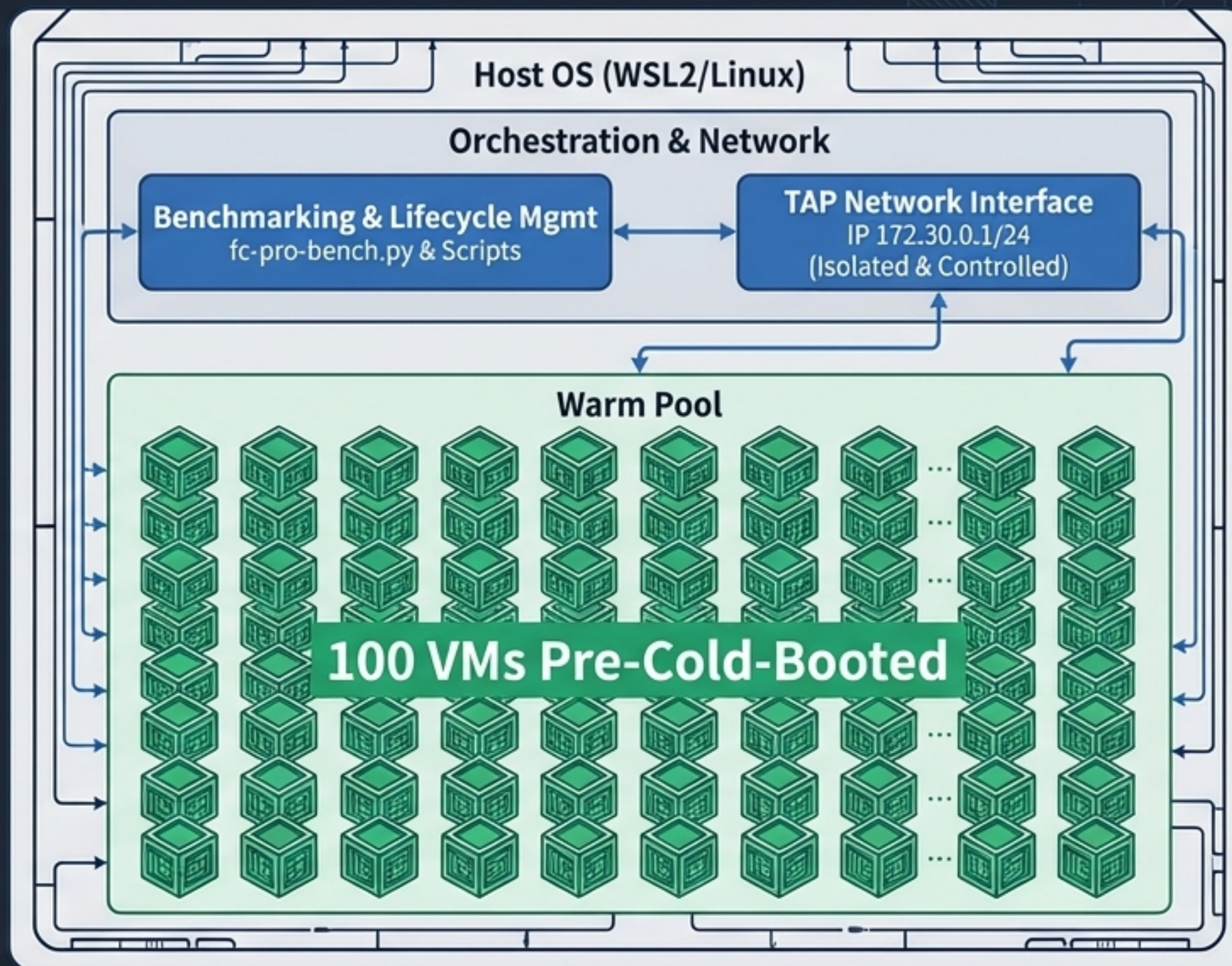
✓ **Networking** : 配置 TAP 網路介面 (預設 IP 172.30.0.1/24)，隔離且可控



✓ **Orchestration** : 專用基準測試腳本 (fc-pro-bench.py) 與生命週期管理 Scripts



✓ **Warm Pool** : 實作並測試預先冷啟動 100 台 VM 待命的「暖池」機制



我們建立了一個高度仿真的生產環境雛形，針對啟動延遲、併發能力與記憶體消耗進行了嚴格的壓力測試。

POC 測試結果：極致的啟動速度與乘載密度

141.50 ms

API 啟動指令延遲 (Average)

證明無伺服器等級的
瞬時反應能力

215.90 ms

長尾延遲 (P99 Latency)

高併發下效能抖動
Jitter 極小

~19.52 MB

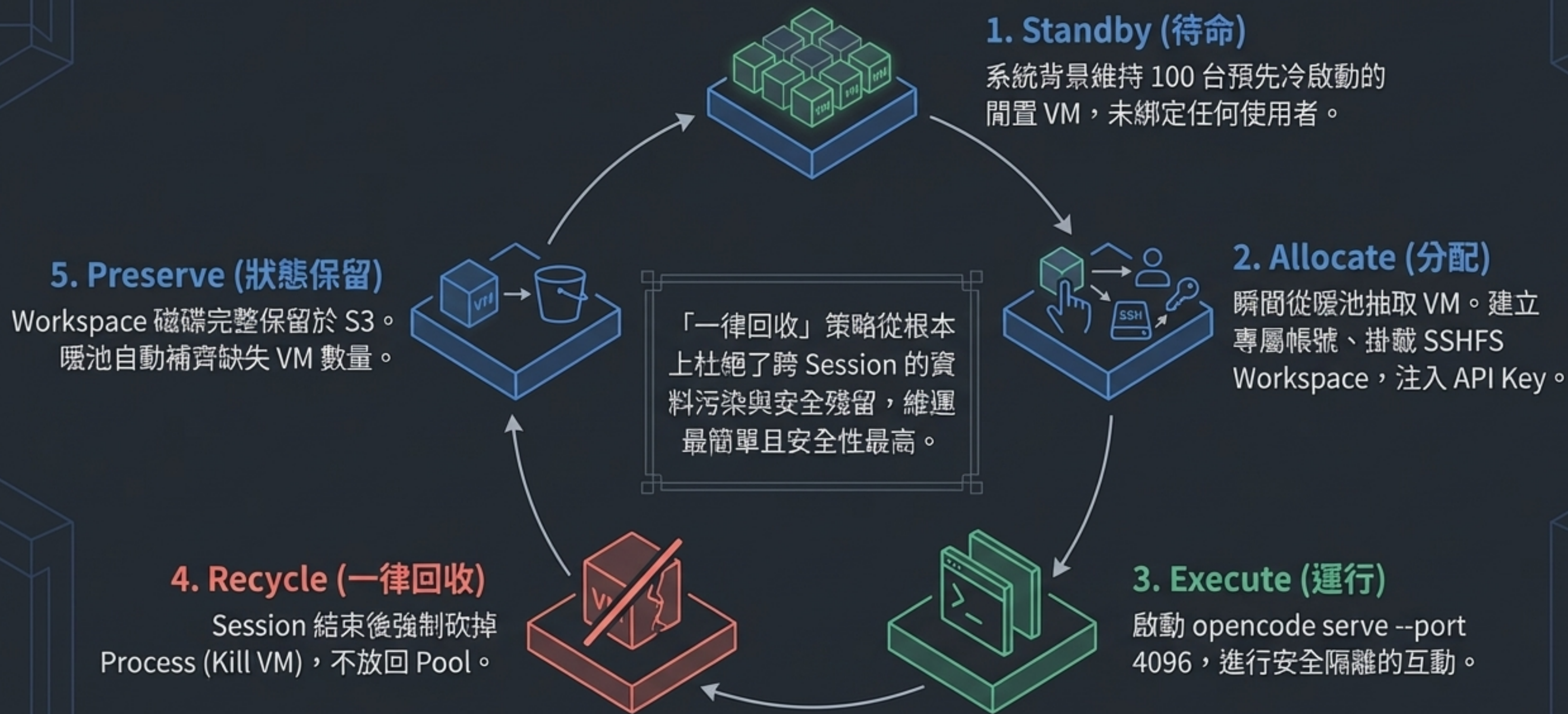
每台 VM 記憶體開銷

極致的基礎架構輕量化

擴展性推演 (Density Projection)

根據 POC 實測數據推算，在配備常規 32GB RAM 的單一節點上，乘載 **1000 個**以上的併發 AI Agent VM 在數學與實務上是完全可行的。這展現了壓倒性的雲端經濟效益。

營運實踐：Warm Pool 調度與生命週期管理



通訊層級剖析：控制面與資料面的徹底隔離

控制面 (Control Plane)

Tech:
Unix Domain Socket API
(firecracker.socket)

Role:
Host 端的 Orchestrator
專用。發送 REST 指令配置
vCPU、內存、網絡與硬碟
生命週期。

Security:
受 Jailer 嚴格限制，Guest
完全無法感知或觸及。



資料面 (Data Plane)

Tech:
virtio-vsock
(以及 TAP 網路 API)

Role:
跨越隔離邊界。
讓 Host 與 Guest 內的
OpenCode 進行 JSON
數據交換，
達成高吞吐低延遲。



控制與資料分離的設計，是確保即使 AI Agent 暴走，也絕對無法干涉底層基礎設施的核心保障。

GO

戰略決策與後續行動建議

最終決議 (Final Verdict)

本次 POC 成功驗證了 Firecracker + MinIO 架構能在滿足極致安全隔離的同時，提供優異的啟動速度與單節點 1000+ VM 的極高密度。建議正式批准此架構進入下一開發階段，無須考慮退回 K8s 路線。

Phase 1

完善 SSHFS 與 Base RootFS 的自動化掛載腳本整合

Phase 2

實作控制平面 (Control Plane)，將現有 Python 腳本轉化為具備生產等級的 Orchestrator API

Phase 3

執行端對端 (E2E) 負載測試，模擬多租戶同時啟動 OpenCode 並讀寫 S3 Workspace 的真實流量

在安全與效能之間，我們已成功找到不需妥協的完美解法。